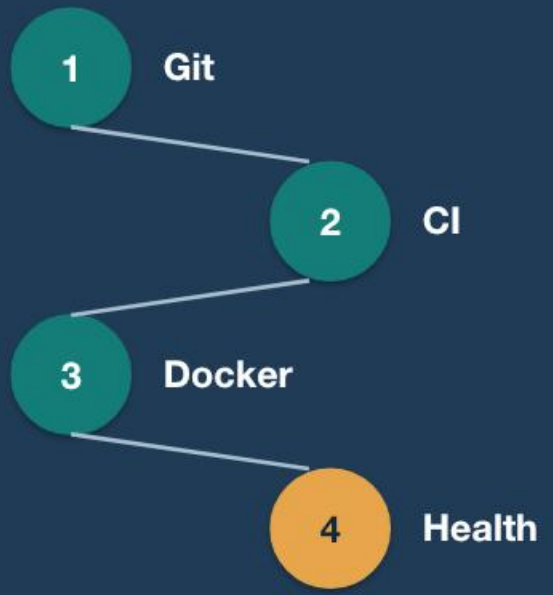
ASSIGNMENT 5

DevOps & CI/CD for Research Projects

Automating a reproducible software workflow with Docker and Jenkins

ReviewGuard | Python research API

Dias Kazikhanov



A working CI demonstration, not a claim of scientific validity

Research results depend on more than source code

A result is produced by a whole computational setup. If any part is missing, another researcher may not be able to rebuild the same workflow.

CODE

source revision
& tests

DATA

dataset version
& provenance

CONFIG

preprocessing
& parameters

RUNTIME

packages, weights
& hardware

DevOps makes the software steps visible and repeatable.

It records and checks how code is built and run. It does not decide whether a dataset, label scheme, or research claim is scientifically sound.

DevOps is a shared way of working

The goal is a short, reliable feedback loop from a proposed change to a tested, runnable artifact.

01	**Version every change**	Keep code, pipeline definitions, and configuration in reviewable history.
02	**Automate routine checks**	Run the same tests and builds after every change.
03	**Describe environments**	Make runtime and deployment setup explicit instead of relying on a laptop state.
04	**Learn from failures**	Use fast, visible feedback to fix defects near the change that caused them.

For research teams: fewer environment surprises, earlier regression detection, easier handoffs, and clearer provenance.

CI/CD turns each change into a repeatable check

Continuous integration (CI) and delivery (CD) share a pipeline, but they answer different questions.



CI Integrate small changes and verify them automatically.

CD Keep a tested version ready to release; deployment may remain gated.

In research software, a green pipeline proves the specified checks passed. It does not validate the scientific conclusion.

ReviewGuard pipeline: from Git to a health-checked container

The Jenkinsfile is versioned with the Python API and runs three operational stages after checkout.



GitHub Actions also runs the full test suite on pushes and pull requests; Jenkins adds a local, Docker-aware demonstration path.

Data and model checkpoints stay outside the image. Mount the trained checkpoint at /models/latest when inference is required.

Docker packages the API runtime, not the research evidence

The image makes the application easier to start consistently across a laptop, CI runner, and test host.

Python 3.12 slim base
small, declared runtime

ReviewGuard package + Uvicorn
API listens on port 8000

Non-root runtime user
UID 10001 · explicit health check

→

Container starts → /health responds

KEEP OUT OF THE IMAGE

- Raw and processed datasets
- Trained model weights
- Local reports and development files

CI uses INSTALL_INFERENCE=0 for a smaller startup image. The default image installs inference dependencies; it still needs a checkpoint mounted separately.

A container controls application packaging. Dataset versions, model provenance, seeds, and hardware still need their own records.

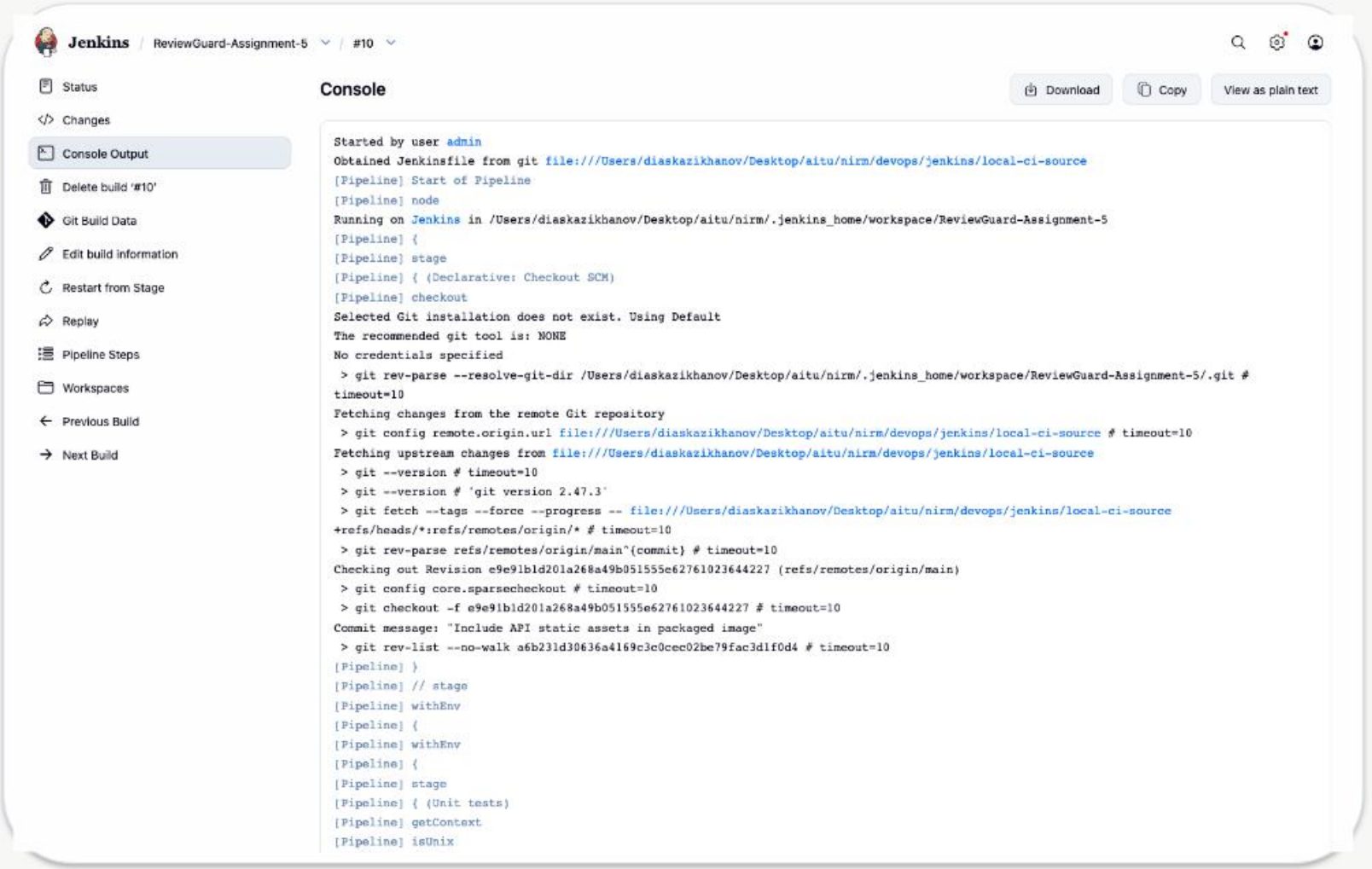
Jenkins shows each stage and its result



BUILD #10 **Checkout → unit tests → Docker image → container health check → post actions**

Builds #10 and #11 are green. The earlier red rows show setup iterations visible in the same local Jenkins history.

Build #10 started from the local SCM snapshot



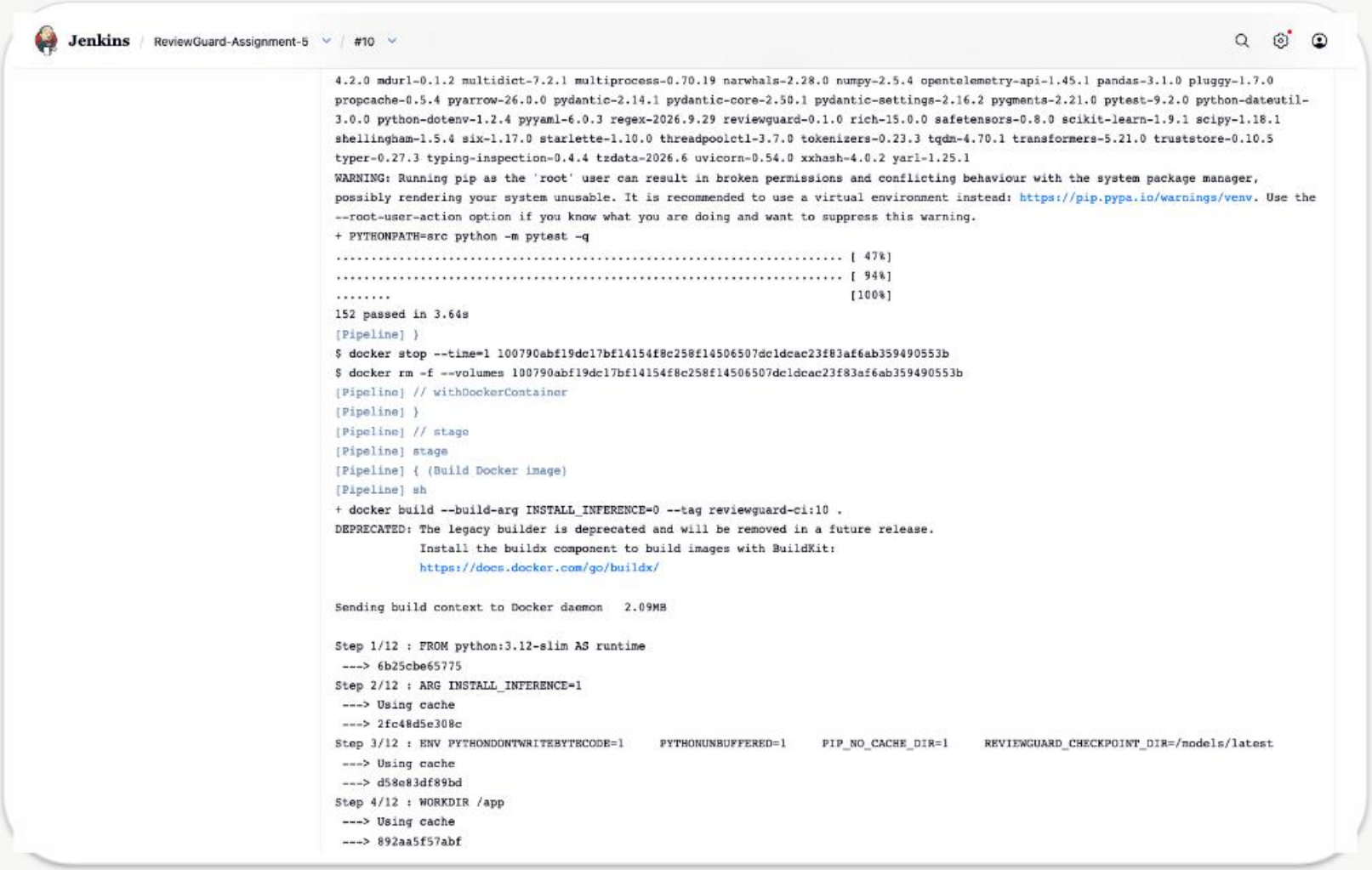
SOURCE REVISION

e9e91b1

The demonstration ran against a temporary local Git snapshot while Assignment 5 files were being prepared.

The completed deliverables are now published on the repository's devops branch. This screenshot records the local demonstration run, not a post-push webhook run.

Automated tests give fast, repeatable feedback



152

tests passed

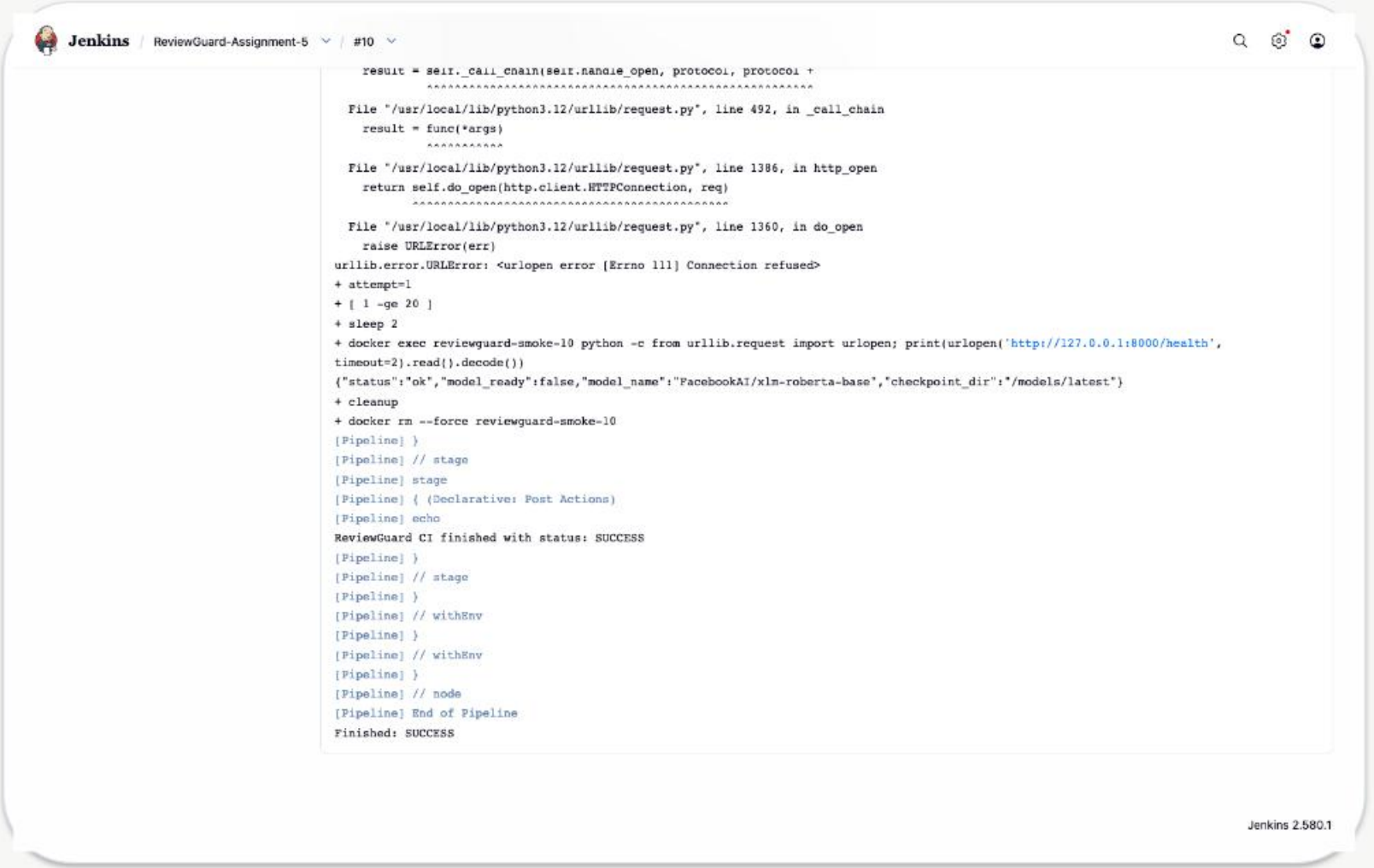
3.64 s

test stage duration

The Docker image build begins after pytest passes.

Evidence: Jenkins console output for build #10.

The container starts and answers its health check



IMAGE

reviewguard-ci:10

HEALTH RESPONSE

status: "ok"
model_ready: false

The initial request raced container startup; a retry returned the response and the pipeline finished SUCCESS.

Liveness verified; no trained checkpoint was loaded.

Operational checks are not scientific validation

The pipeline provides evidence about software behavior at one recorded revision.

WHAT THE RUN SUPPORTS

- The specified test suite passed on the recorded source snapshot.
- The Docker image built from the declared project files.
- The API process started and /health returned a response.

WHAT IT DOES NOT ESTABLISH

- Model accuracy or generalization to new review sources.
- Dataset representativeness or label validity.
- Bitwise reproducibility across hardware and runtimes.

Research quality still depends on provenance for datasets, checkpoints, configuration, seeds, and source revisions.

Implementation exposed integration issues early

A useful pipeline does more than automate the happy path. It reveals mismatches between packaging, the runner, and the application.

ISSUE	FIX / DESIGN CHOICE
No active Docker daemon	Start a local Docker runtime before building images.
Jenkins agent lacked Docker CLI / Git	Add required tools to the Jenkins image and Python test agent.
API HTML asset missing from package	Declare static HTML as package data; rerun the health check.
CPU-only CI runner	Install CPU PyTorch wheels to avoid unused CUDA downloads.
Container startup race	Retry /health briefly and always remove the temporary smoke container.

The health check now tests a real built container rather than only importing the API package.

Kubernetes is a deployment option, not a requirement

The assignment's working scope is Docker + Jenkins CI. Registry publishing and cluster deployment remain optional because no credentials or target cluster are configured.

A PRACTICAL NEXT PATH

- Push a versioned image to a registry with scoped credentials.
- Add a webhook or multibranch job for the published devops branch.
- Deploy to a test environment and keep promotion gated.
- Use Kubernetes when shared scheduling or multiple services justify it.

TAKEAWAY

DevOps makes research software easier to test, package, and rerun.

Use automation to strengthen traceability and reliability, while keeping scientific claims tied to data and empirical evidence.

Repository: github.com/Svoppy/review-api/tree/devops

References

1. **Wilson et al. (2014). Best Practices for Scientific Computing. PLoS Biology 12(1): e1001745.**

   https://doi.org/10.1371/journal.pbio.1001745

2. **Osborne et al. (2014). Ten simple rules for effective computational research. PLoS Comput. Biol. 10(1): e1003506.**

   https://doi.org/10.1371/journal.pcbi.1003506

3. DORA. Continuous Integration capability.

   https://dora.dev/capabilities/continuous-integration/

4. DORA. Continuous Delivery capability.

   https://dora.dev/capabilities/continuous-delivery/

5. Docker. Python language-specific guide.

   https://docs.docker.com/guides/python/

6. Jenkins. Pipeline and Pipeline as Code documentation.

   https://www.jenkins.io/doc/book/pipeline/

7. Kubernetes. Concepts documentation.

   https://kubernetes.io/docs/concepts/